In-Memory Data Structure for Scalable Distributed Applications Using Redis

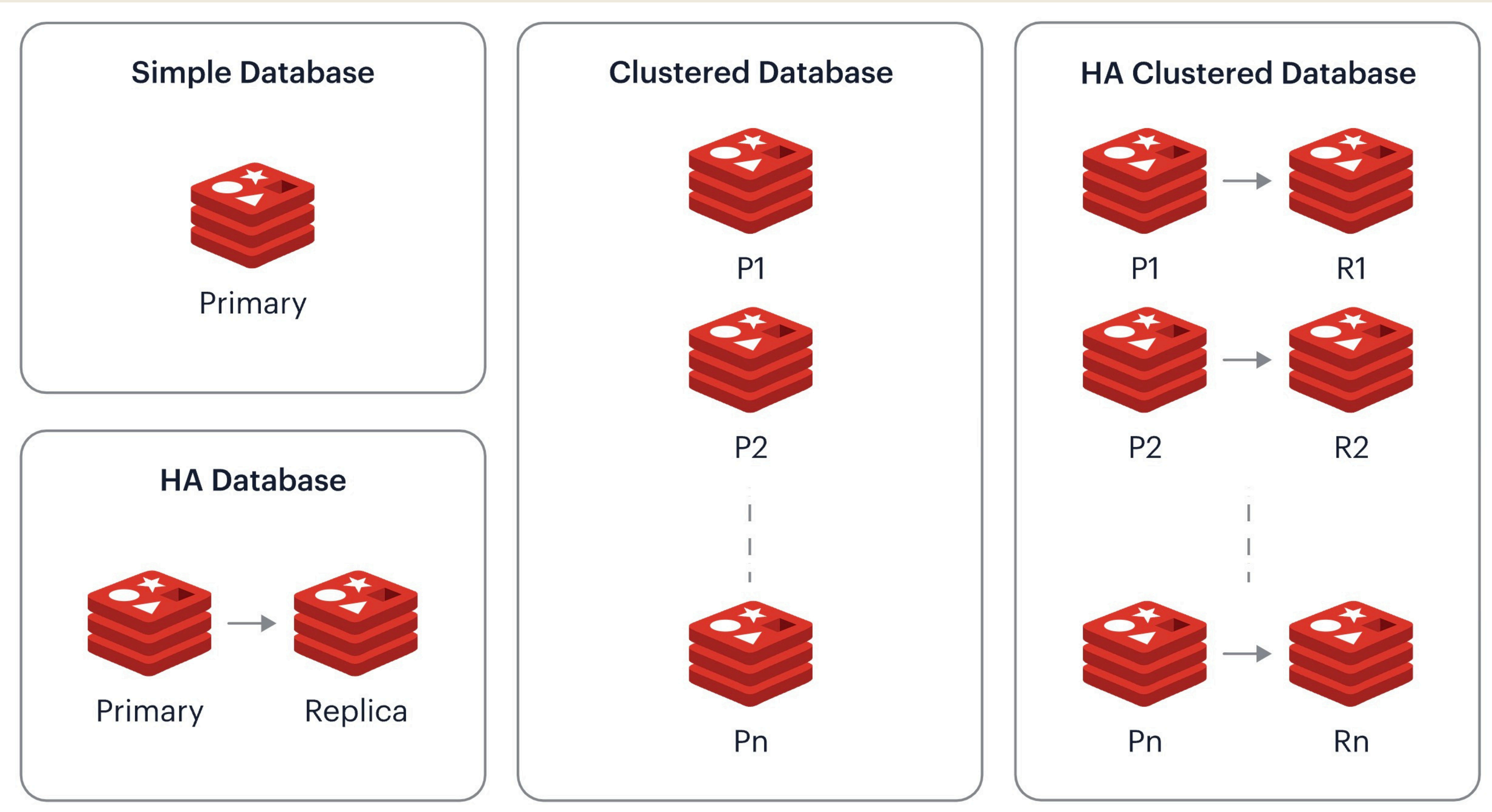
AUTHORS

Imane Ait Yahiatene
Gerson Gerard Cruz
Shrijan Subedi
Sidorela Uku

AFFILIATIONS

Erasmus Mundus Green Networking and Cloud Computing (GENIAL) Program
Luleå University of Technology

Demonstrating multiple use case scenarios of in-memory data storage using Redis



01. Introduction

- Traditional data storage systems are disk-based and often face latency when managing client states in distributed environments. Whereas In-memory data structures provide efficient synchronization of distributed clients in real-time systems.
- Redis is an open-source in-memory data structure store. Its features include publish-subscribe messaging, atomic operations, and distributed locks. It is ideal for real-time synchronization in applications like collaborative tools, and IoT systems.

02. Literature review questions

- How do in-memory data structures improve the performance of synchronization mechanisms in distributed systems compared to traditional storage solutions?
- How is Redis used to implement the publish-subscribe model, and what are its strengths and limitations for real-time messaging and synchronization of multiple clients?
- How can redis improve the scalability of real-time distributed applications in terms of multiple metrics–latency, throughput, and response time?

03. Redis as a Synchronization Mechanism

Redis operates entirely in RAM, enabling high-speed data access with minimal latency, suitable for real-time applications like IoT systems (Sanfilippo, 2013).

- Its key data structures are:
  - **Strings**: Store key-value pairs for client states.
  - **Lists:** Manage event queues.
  - **Sets:** Track unique elements, useful for leaderboards.
  - **Hashes:** Efficient for structured data like user sessions
  - Atomic operations ensure consistency in multi-client environments.
- **Publish-subscribe model:(Sanfilippo, 2018).**

Real-time messaging model where clients subscribe to channels and receive updates from publishers without polling. It is suitable for chat systems, collaborative tools, and real-time notifications. Redis streams offers persistence for message delivery when clients are offline

- **Distributed Locking (Redlock):**

Prevents race conditions by using atomic operations to ensure only one client modifies shared data at a time, critical for consistency in distributed systems

- **Challenge:**

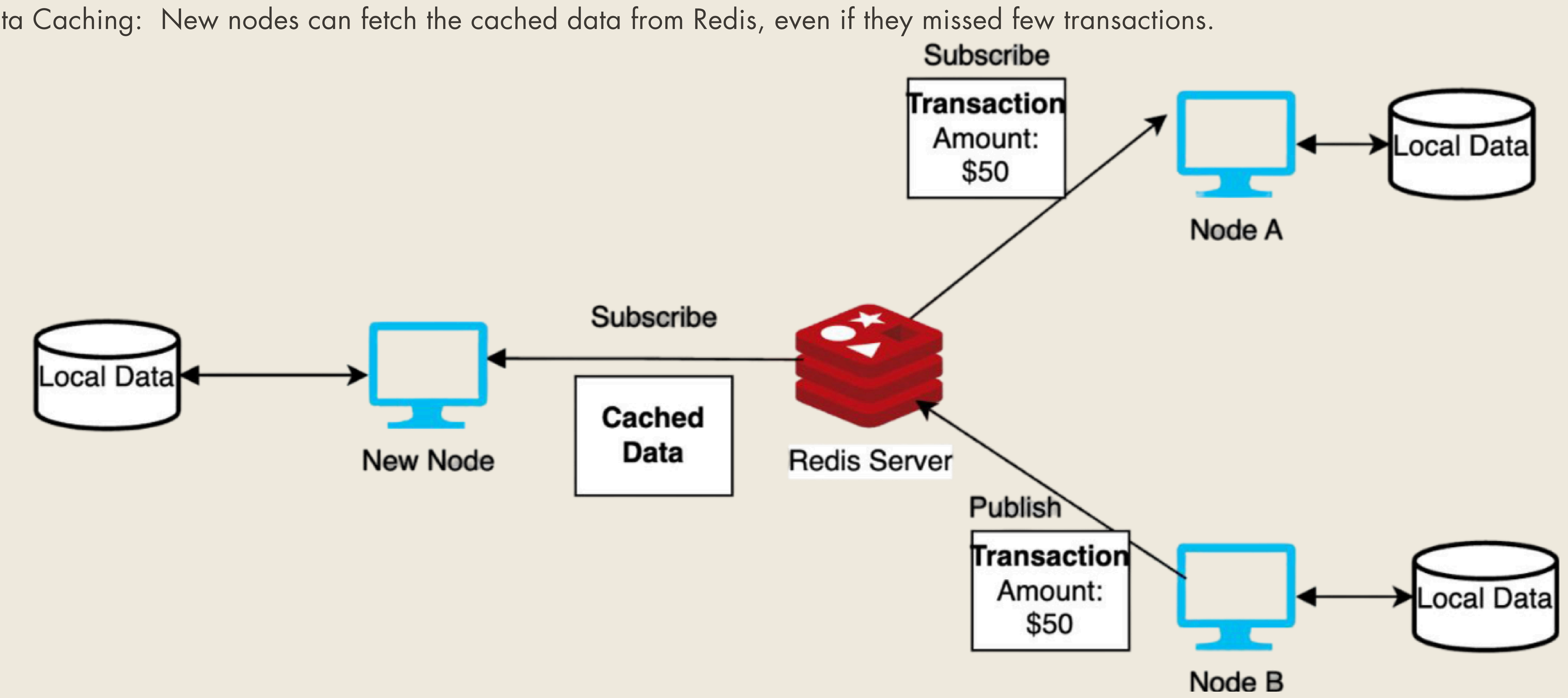
Potential consistency issues in geographically distributed systems due to replication delays.

04. Pub-Sub Model for Synchronization

Pub/Sub is a messaging model that allows different components in a distributed system to communicate with one another by publishing and subscribing to messages. This pattern by providess a way to synchronize data among distributed nodes, like bank branches and version control.

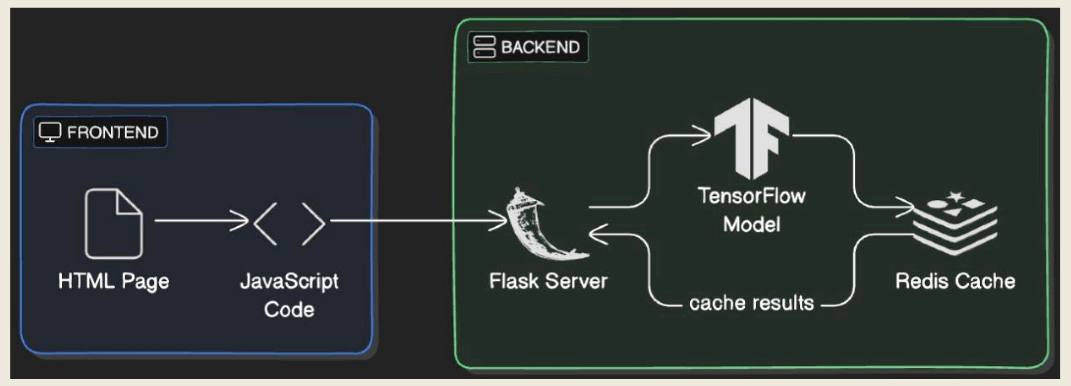
Synchronization of Distributed Clients is achieved through:

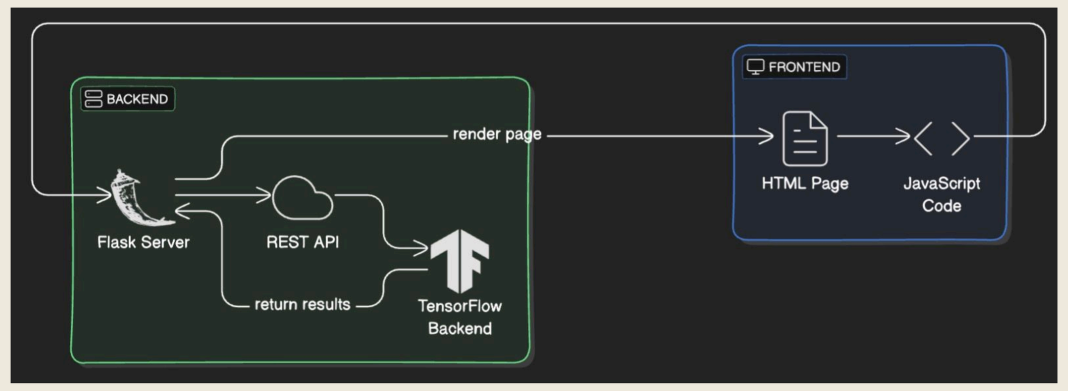
- Event-Driven Updates: The publisher broadcasts events to all channel subscribers keeping the nodes synchronized.
- Decoupled Communication: Clients are not directly connected to each other but are synchronized via Redis.
- Real-Time Synchronization: Event-driven messages are delivered immediately, ensuring real-time updates.
- Data Caching: New nodes can fetch the cached data from Redis, even if they missed few transactions.



05. Efficient scalability

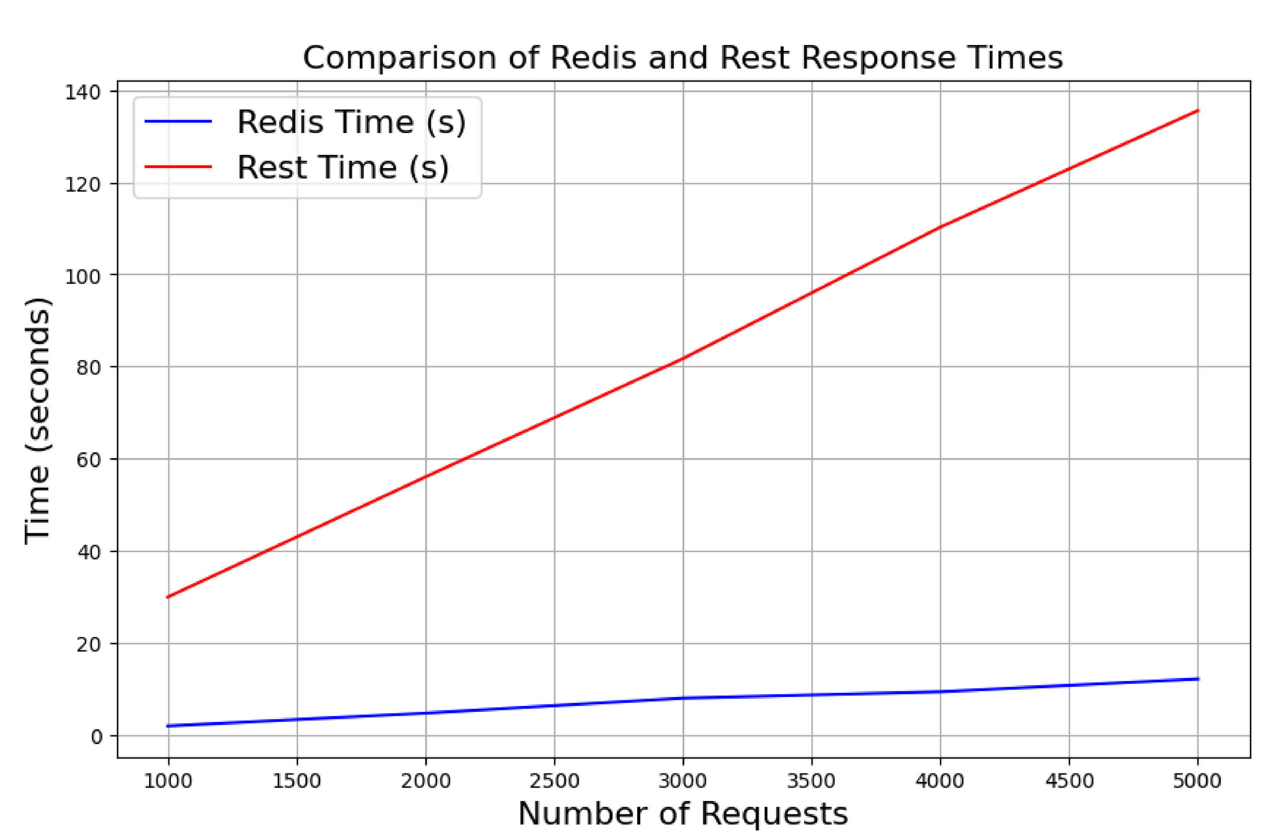
Redis excels in efficiency and scalability due to its in-memory data storage, enabling ultra-fast read and write operations with the support of caching. Its ability to handle high volumes of concurrent requests with minimal latency makes it ideal for real-time applications.

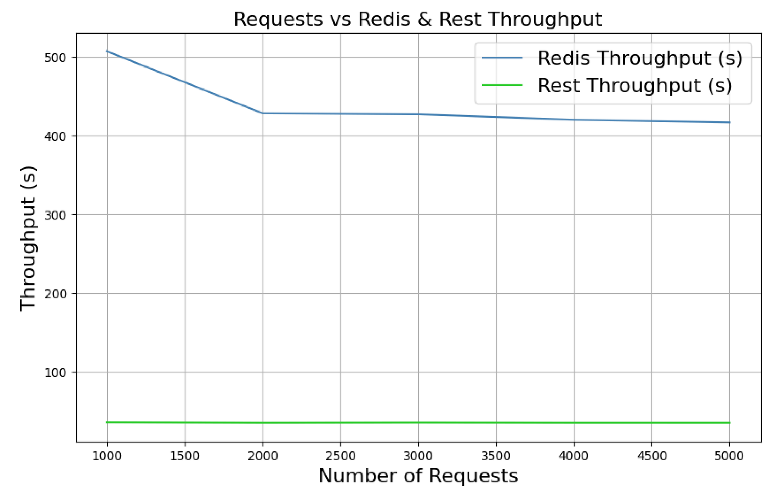


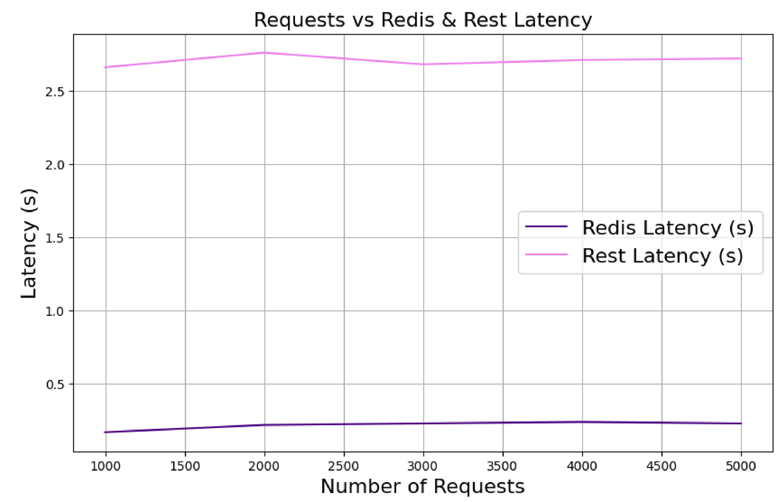


06. Results

The data shows that Redis significantly outperforms Rest across all metrics. Redis handles requests faster, with much lower latency, and maintains high throughput as the number of requests increases. In contrast, Rest experiences much slower response times, higher latency, and struggles with a lower throughput that remains consistent, indicating less efficient handling of increasing load. Overall, Redis scales better and is far more efficient under higher demand compared to Rest making it more suitable for real-time and distributed applications.







07. Conclusion

In conclusion, Redis proves to be an effective solution for real-time and distributed applications due to its in-memory data storage and scalability. Its efficient pub/sub handling mechanism enables real-time updates crucial for industry-based applications. It consistently handles high request volumes with minimal latency and significantly better throughput compared to traditional Rest-based systems. This makes Redis particularly suited for use cases requiring efficient synchronization, such as collaborative tools, IoT systems, and other applications where performance under load is critical.

Related Literature

Scan the QR code to see references